

Python Loops – Complete Notes

1. What are Loops?

A loop is used to execute a block of code repeatedly until a condition becomes false or until all items in a sequence are processed.

Why Use Loops?

Without loops:

```
print("Welcome")
print("Welcome")
print("Welcome")
print("Welcome")
print("Welcome")
```

With loops:

```
for i in range(5):
    print("Welcome")
```

Advantages

- Reduces code duplication
- Saves time
- Makes programs easier to maintain
- Useful for repetitive tasks

2. Types of Loops in Python

Python provides two main loops:

1. For Loop
2. While Loop

For Loop

A for loop is used when the number of iterations is known or when iterating through a sequence.

Syntax

```
for variable in sequence:  
    statements
```

Example 1: Display Numbers

```
for num in [2, 4, 6, 8, 10]:  
    print(num)
```

Output

```
2  
4  
6  
8  
10
```

Example 2: Iterate Through a String

```
for ch in "PYTHON":  
    print(ch)
```

Output

```
P  
Y  
T  
H  
O  
N
```

Example 3: Display Student Names

```
students = ["Ravi", "Anu", "Kiran"]
```

```
for name in students:  
    print(name)
```

Output

```
Ravi  
Anu  
Kiran
```

Example 4: Calculate Total Marks

```
marks = [78, 82, 91, 67]
```

```
total = 0
```

```
for mark in marks:  
    total += mark
```

```
print("Total:", total)
```

Output

Total: 318

range() Function

The range() function generates a sequence of numbers.

range(stop)

```
for i in range(5):  
    print(i)
```

Output

0
1
2
3
4

range(start, stop)

```
for i in range(3, 8):  
    print(i)
```

Output

3
4
5
6
7

range(start, stop, step)

```
for i in range(5, 26, 5):  
    print(i)
```

Output

```
5  
10  
15  
20  
25
```

Reverse Order

```
for i in range(10, 0, -2):  
    print(i)
```

Output

```
10  
8  
6  
4  
2
```

3.While Loop

A while loop executes as long as a condition remains True.

Syntax

```
while condition:  
    statements
```

Example 1: Print Numbers

```
num = 1  
  
while num <= 5:  
    print(num)  
    num += 1
```

Output

```
1
2
3
4
5
```

Example 2: Countdown Timer

```
count = 5

while count > 0:
    print(count)
    count -= 1

print("Start")
```

Output

```
5
4
3
2
1
Start
```

Example 3: Sum of Numbers

```
num = 1
total = 0

while num <= 4:
    total += num
    num += 1

print(total)
```

Output

```
10
```

Example 4: Multiplication Table

```
n = 4
i = 1

while i <= 10:
    print(n, "x", i, "=", n*i)
    i += 1
```

Output

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
...
4 x 10 = 40
```

Difference Between For Loop and While Loop

For Loop	While Loop
Used when iterations are known	Used when iterations are unknown
Simpler syntax	Condition must be managed manually
Best for sequences	Best for condition-based repetition

Example

For Loop

```
for i in range(7):
    print(i)
```

While Loop

```
i = 0

while i < 7:
    print(i)
    i += 1
```

4. Nested Loops

A loop inside another loop is called a Nested Loop.

Syntax

```
for outer in range():
    for inner in range():
        statements
```

Example 1: Rectangle Pattern

```
for row in range(3):
    for col in range(4):
        print("*", end=" ")
    print()
```

Output

```
* * * *
* * * *
* * * *
```

Example 2: Number Grid

```
for row in range(3):
    for col in range(3):
        print(col + 1, end=" ")
    print()
```

Output

```
1 2 3
1 2 3
1 2 3
```

5. Loop Control Statements

Loop control statements alter the normal flow of a loop.

Python provides:

1. break
2. continue
3. pass

break Statement

The break statement immediately terminates the loop.

Example 1

```
for num in range(1, 11):
    if num == 6:
        break

    print(num)
```

Output

```
1
2
3
4
5
```

Example 2

```
password = "python123"

while True:
    user_input = input("Enter Password: ")

    if user_input == password:
        print("Access Granted")
        break

    print("Wrong Password")
```

continue Statement

The continue statement skips the current iteration and moves to the next iteration.

Example 1

```
for num in range(1, 8):

    if num == 4:
        continue

    print(num)
```

Output

```
1
2
```



```
3
5
6
7
```

Example 2

Display only odd numbers.

```
for num in range(1, 11):
```

```
    if num % 2 == 0:
        continue
```

```
    print(num)
```

Output

```
1
3
5
7
9
```

pass Statement

The pass statement does nothing.

It is used as a placeholder when code will be written later.

Example 1

```
for i in range(5):
    pass
```

```
print("Program Executed")
```

Output

```
Program Executed
```

Example 2

```
if True:
```

```
    pass
else:
    print("Hello")
```

Infinite Loop

A loop that never ends is called an infinite loop.

Example

```
while True:
    print("Running...")
```

Output

```
Running...
Running...
Running...
...
```

Loop with Else

Python allows an else block with loops.

The else block executes if the loop completes normally.

Example 1

```
for i in range(1, 6):
    print(i)
else:
    print("Loop Completed")
```

Output

```
1
2
3
4
5
Loop Completed
```

Example 2

```
for i in range(1, 6):  
  
    if i == 4:  
        break  
  
    print(i)  
  
else:  
    print("Completed")
```

Output

```
1  
2  
3
```

(else does not execute because break terminated the loop.)

Real-Time Uses of Loops

Reading Employee Records

```
employees = ["Raj", "Anu", "Sai"]  
  
for emp in employees:  
    print(emp)
```

ATM PIN Validation

```
attempts = 3  
  
while attempts > 0:  
  
    pin = input("Enter PIN: ")  
  
    if pin == "1234":  
        print("Welcome")  
        break  
  
    attempts -= 1
```

Processing Orders

```
orders = [101, 102, 103, 104]
```

```
for order in orders:  
    print("Processing Order:", order)
```